



Reflection and Traceability Report on RoCam

Team #3, SpaceY
Zifan Si
Jianqing Liu
Mike Chen
Xiaotian Lou

Contents

1	Changes in Response to Feedback	2
1.1	SRS and Hazard Analysis	2
1.2	Design and Design Documentation	3
1.3	VnV Plan and Report	3
1.3.1	VnV Plan	3
1.3.2	VnV Report	3
1.3.3	Peer Review Responses	4
2	Extras	4
3	Design Iteration (LO11 (PrototypeIterate))	5
4	Design Decisions (LO12)	5
5	Economic Considerations (LO23)	7
6	Reflection on Project Management (LO24)	7
6.1	How Does Your Project Management Compare to Your Development Plan	7
6.2	What Went Well?	7
6.3	What Went Wrong?	8
6.4	What Would you Do Differently Next Time?	8
7	Ethics and Equity (LO18)	8
8	Reflection on Capstone	9
8.1	Which Courses Were Relevant	9
8.2	Knowledge/Skills Outside of Courses	9

1 Changes in Response to Feedback

This section summarizes all changes made to project deliverables in response to feedback from TAs, the instructor, peer reviewers, the project supervisor (Dr. Shahin Sirouspour), and the client (McMaster Rocketry Team). Our team used GitHub Issues and pull requests to track every item of feedback and the corresponding changes. The primary commit addressing documentation feedback is 30cd72c (PR #365), with additional changes in commit 45a7df6.

Our supervisor, Dr. Sirouspour, provided feedback throughout the project, particularly after the Rev 0 demonstration. His feedback emphasized the importance of robust error handling in the gimbal control loop and the need for a clear emergency stop mechanism. This feedback directly led to the addition of safety requirement SR4 (Emergency Stop) in the Hazard Analysis and the corresponding traceability entries in the SRS.

1.1 SRS and Hazard Analysis

Feedback was received from TA Tanya (Issue #107) via green-highlighted annotations, as well as from peer review. All items have been addressed in commit 30cd72c (PR #365).

Item	Source	Feedback	Response
HA-1	TA Tanya	Placeholder “Date1/Date2” remained in the Hazard Analysis document	Replaced with actual revision dates
HA-2	TA Tanya	Typo: “a existing”	Corrected to “an existing”
HA-3	TA Tanya	Typo: “moodule”	Corrected to “module”
HA-4	TA Tanya	Typo: “a off the shelf”	Corrected to “an off-the-shelf”
HA-5	TA Tanya	Typo: “accelerated”	Corrected to “accelerated”
HA-6	TA Tanya	Typo: “excesses”	Corrected to “exceeds”
HA-7	TA Tanya	Inconsistent phrasing: “general of the shelf”	Corrected to “Off-the-Shelf” with consistent capitalization
HA-8	TA Tanya	Safety requirements SR1–SR3 lacked sufficient detail	Expanded SR1–SR3 with specific conditions, thresholds, and verification methods
HA-9	Supervisor	No emergency stop mechanism defined	Added SR4 (Emergency Stop) requirement specifying hardware and software stop triggers
HA-10	TA Tanya	No traceability between HA and SRS	Added a full HA ↔ SRS traceability table mapping each hazard to corresponding safety requirements
SRS-1	TA Tanya	Typo: “equipments”	Corrected to “equipment”
SRS-2	TA Tanya	Typo: “Through out”	Corrected to “Throughout”
SRS-3	TA Tanya	Formatting: “500CAD” missing space	Corrected to “500 CAD”
SRS-4	TA Tanya	Duplicate identifier AR-1 used for two different requirements	Renamed the second instance to ACR-1 to eliminate ambiguity
SRS-5	TA Tanya	Safety requirements section lacked HA traceability	Added Hazard Analysis trace references to all safety requirements in the SRS

1.2 Design and Design Documentation

Feedback was received from Issues #164 and #240. All items have been addressed in commit 30cd72c (PR #365).

Item	Source	Feedback	Response
MG-1	TA (Issue #164)	Typo: “relys”	Corrected to “relies on”
MG-2	TA (Issue #164)	Typo: “Stablize”	Corrected to “Stabilize”
MG-3	TA (Issue #164)	Residual <code>\wss{}</code> template comments remained in the Module Guide	Removed all <code>\wss{}</code> comments from the MG
MIS-1	TA (Issue #240)	Incorrect year: “Jan 2025” should be “Jan 2026”	Corrected to “Jan 2026”
MIS-2	TA (Issue #240)	Typo: “presistent”	Corrected to “persistent”
MIS-3	TA (Issue #240)	Residual <code>\wss{}</code> template comments remained in the MIS	Removed all <code>\wss{}</code> comments from the MIS

1.3 VnV Plan and Report

Feedback was received from Issues #122, #337, #342–#348, #148, and #149. Changes were made in commits 30cd72c and 45a7df6 (PR #365).

1.3.1 VnV Plan

Item	Source	Feedback	Response
VP-1	TA (Issue #122)	Residual <code>\wss{}</code> template comments in VnV Plan	Removed all <code>\wss{}</code> comments
VP-2	TA (Issue #122)	No unit test summary section	Added a dedicated unit test summary section listing all unit tests and their status
VP-3	TA (Issue #122)	Grammar: “a outline”	Corrected to “an outline”
VP-4	TA (Issue #337)	Test-tracking details were insufficient	Enhanced test-tracking details with specific pass/fail criteria and expected timelines
VP-5	Issue #148	Error-handling tests lacked timing specifications	Added timing constraints to error-handling tests (e.g., error detection within 200 ms)

1.3.2 VnV Report

Item	Source	Feedback	Response
VR-1	Peer (Issue #346)	Green “Pass” text is not distinguishable for colour-blind users	Changed green “Pass” indicators to teal (WCAG AA compliant, $\geq 4.5:1$ contrast ratio)
VR-2	TA (Issue #337)	VnV Report scope was unclear	Added explicit scope clarification at the beginning of the report

Item	Source	Feedback	Response
VR-3	Peer (Issue #348)	No discussion of planned future improvements	Added a “Planned Future Improvements” section
VR-4	Peer (Issue #344)	Duplicate descriptions between VnV Plan and VnV Report	Condensed overlapping descriptions and added cross-references to the VnV Plan

1.3.3 Peer Review Responses

Issue	Feedback	Response
#342	Usability testing should be included	Out of scope for this deliverable; usability was addressed through iterative client demos and informal user testing with the McMaster Rocketry Team. Explained in issue.
#343	Table formatting issues in VnV Report	Already fixed prior to peer review being filed; confirmed and closed the issue
#344	Duplicate test descriptions between Plan and Report	Condensed duplicated content and replaced with cross-references to the VnV Plan
#345	Hardware testing scope was unclear	Added clarification that hardware-specific tests (e.g., motor stress tests) are outside the scope of the software VnV
#346	Green text for “Pass” is inaccessible	Changed to teal with a contrast ratio $\geq 4.5:1$ for WCAG AA compliance
#347	Some tests lacked sufficient detail	Enhanced test descriptions with explicit inputs, expected outputs, and pass/fail criteria
#348	No section on future improvements	Added “Planned Future Improvements” section covering field testing, expanded model training, and weather-proofing
#148	Error-handling tests lacked timing information	Added specific timing thresholds (e.g., error detection within 200 ms, recovery within 1 s)
#149	Field testing not covered in VnV	Explicitly noted that field testing is out of scope for the current VnV cycle; listed as a planned future improvement

2 Extras

Our team completed two extras as part of this capstone project, both of which were approved by the course instructor in our problem statement.

- **Circuit Design Report:** Documents the design and verification of a custom STM32-based printed circuit board (PCB) for gimbal motor control. The PCB interfaces with the Jetson Orin Nano via UART and provides precise two-axis servo control with encoder feedback. The report includes schematic design, component selection rationale, PCB layout considerations, and hardware testing results. Located in `docs/Extras/CircuitDesign/`.
- **Machine Learning Report:** Documents the YOLO-based object detection model training pipeline used for rocket detection and tracking. The report covers dataset preparation, model architecture selection, training methodology, TensorRT optimization for deployment on the Jetson Orin Nano,

and benchmark results comparing inference speed and accuracy across different optimization levels. Located in `docs/Extras/MachineLearning/`.

3 Design Iteration (LO11 (PrototypeIterate))

The design of RoCam evolved substantially over the course of the project through multiple iterations driven by feedback from our supervisor, client, and peer reviewers.

Rev 0 (January 2026): The initial design established the core module decomposition (M1–M21) covering computer vision, livestream management, transcoding, gimbal control, and the frontend interface. The preliminary FMEA identified basic failure modes, and safety requirements SR1–SR3 addressed overheating, connection loss, and power supply failures. At this stage, the system used a direct single-process architecture for the CV pipeline, which we quickly found could not meet the real-time performance requirements.

Feedback-driven changes: Peer review and TA feedback (Issues #107, #122, #164, #240, #337) identified significant gaps. Traceability between the Hazard Analysis and SRS was missing entirely. Safety requirements lacked specificity. Several modules had incomplete interface specifications. Our supervisor, Dr. Sirouspour, emphasized the need for an emergency stop mechanism after observing the Rev 0 demonstration, noting that the gimbal could pose a safety risk if the control loop failed.

Rev 1 (April 2026): Incorporating all feedback, Rev 1 introduced several key improvements:

- Added SR4 (Emergency Stop) with both hardware and software triggers, including a physical kill switch and a software-initiated stop command accessible from the UI.
- Created a full HA ↔ SRS traceability matrix ensuring every identified hazard maps to at least one safety requirement and vice versa.
- Expanded test coverage with a dedicated unit test summary and enhanced integration test specifications.
- Adopted process separation—running CV inference, livestream encoding, and transcoding as independent worker processes—to achieve the target of 1080p at 60 fps with less than 120 ms end-to-end latency.

Client-driven iteration: The McMaster Rocketry Team provided hands-on feedback during iterative demos that shaped several user-facing features:

- **Drag-to-move gimbal control:** The client requested an intuitive way to manually aim the camera when automatic tracking was not engaged. We implemented a click-and-drag interface on the video feed that translates mouse movements into gimbal angle commands.
- **Bilingual UI:** To accommodate team members more comfortable in French, we added English/French language toggle support.
- **Confirmation prompts:** After an incident where a team member accidentally reset the gimbal during a demo, we added confirmation dialogs for all potentially dangerous operations (e.g., gimbal reset, firmware flash).

Hardware iteration: Mid-project, the originally selected camera sensor was found to have insufficient low-light performance for dawn/dusk launches. We changed to a sensor with better sensitivity, which required redesigning the camera mount CAD model and updating latency measurements throughout the documentation. This change validated the importance of our modular hardware abstraction—the software required only a configuration change to accommodate the new sensor.

4 Design Decisions (LO12)

Our design decisions were shaped by the interplay of limitations, assumptions, and constraints specific to the RoCam project.

Limitations

- **Budget constraint (500 CAD):** The total hardware budget of 500 CAD eliminated commercial tracking systems (typically \$5,000+) and constrained our platform choice. The Jetson Orin Nano was selected because it offers GPU-accelerated inference in a compact, power-efficient form factor within our budget. This limitation also drove the decision to design a custom PCB for gimbal control rather than purchasing an expensive commercial servo controller.
- **Power and thermal constraints:** The system must operate outdoors in a field environment powered by a portable battery. This ruled out high-power server-grade GPUs and motivated our investment in TensorRT optimization, which reduced inference power consumption by approximately 40% compared to running the YOLO model without optimization.
- **Network availability:** Launch sites often lack reliable internet connectivity. This drove the decision to run all processing on-device (edge computing) rather than offloading to cloud services, and to use a local Wi-Fi access point for the operator interface.

Assumptions

- **Jetson as deployment platform:** We assumed early in the project that the Jetson Orin Nano would be the final deployment platform. This assumption guided our investment in TensorRT optimization and CUDA-based GStreamer pipelines. The assumption was validated through benchmarking, which confirmed that the Jetson could achieve the required inference speed (≤ 15 ms per frame) with TensorRT.
- **Single-rocket tracking:** We assumed the system would track one rocket at a time, simplifying the detection pipeline to single-object tracking after initial detection. This assumption aligned with the client's use case.
- **Operator presence:** We assumed a human operator would always be present to initiate tracking and intervene if needed, which justified the semi-automatic design rather than a fully autonomous system.

Constraints and Key Decisions

- **Process separation (CV, Livestream, Transcode):** The real-time performance requirement (1080p at 60 fps with < 120 ms latency) could not be met with a single-process architecture. We separated the CV inference, livestream encoding, and transcoding into independent worker processes communicating via shared memory and IPC. This decision was driven entirely by measured performance bottlenecks during Rev 0 testing.
- **Flask + Server-Sent Events (SSE) over WebSocket:** For the communication between backend and frontend, we chose Flask with SSE rather than WebSocket. SSE is simpler to implement and sufficient for our use case, which requires only unidirectional status updates from server to client. Bidirectional commands (e.g., gimbal control) are handled via standard HTTP POST requests. This decision reduced implementation complexity without sacrificing functionality.
- **Custom PCB over off-the-shelf servo controller:** The gimbal requires precise two-axis control with real-time encoder feedback at rates that commodity servo controllers could not reliably provide. The custom STM32-based PCB allowed us to implement a tuned PID control loop with direct encoder integration, achieving smoother and more responsive gimbal movement.
- **React/TypeScript frontend:** We chose React with TypeScript for the operator interface to enable a responsive single-page application that works well on tablets in outdoor settings. TypeScript provided type safety that caught several integration bugs at compile time.

5 Economic Considerations (LO23)

Market analysis: There is currently no commercially available product at a comparable price point that provides automated rocket tracking with live video feed for amateur and collegiate rocketry. Existing solutions either require manual operation (standard tripod-mounted cameras) or cost upwards of \$5,000 (commercial tracking mounts). The model rocketry community in North America includes approximately 50,000 active hobbyists and over 200 university rocketry teams, representing a meaningful niche market.

Cost to produce: Our current hardware bill of materials is approximately 350 CAD, covering the camera sensor, gimbal assembly, Jetson Orin Nano, custom PCB components, and structural hardware. For a production version, additional costs would include a weatherproof enclosure (~50 CAD), a user-friendly setup wizard (software development), and regulatory compliance testing (e.g., FCC for the Wi-Fi module). We estimate the total production cost at approximately 500–600 CAD per unit at low volumes.

Pricing and break-even: We estimate a retail price of 800–1,200 CAD would be competitive and viable. At an average margin of ~350 CAD per unit, approximately 500 units would need to be sold to recover tooling, injection mould, and certification costs (estimated at ~175,000 CAD).

Go-to-market strategy: Given the niche market, an open-source strategy would be most effective. By releasing the software under the MIT license, we can attract contributors, build community trust, and generate organic adoption. Revenue could be generated through:

- Selling pre-assembled hardware kits for users who prefer not to build their own
- Offering premium support and custom integration services for university teams
- Partnering with rocketry suppliers for distribution

6 Reflection on Project Management (LO24)

6.1 How Does Your Project Management Compare to Your Development Plan

We largely followed our Development Plan as written. Weekly team meetings were held consistently, with agendas and minutes tracked via GitHub Issues. Asynchronous communication was conducted on Discord, and bi-weekly meetings with our supervisor, Dr. Sirouspour, were held over Zoom. Team member roles aligned with the plan: Zifan Si led frontend development, Jianqing Liu handled backend architecture and served as team lead, Mike Chen was responsible for the computer vision pipeline, and Xiaotian Lou managed hardware integration and PCB design.

We used GitHub Projects for task tracking, with a Kanban-style board organized by sprint. All code changes went through pull request review before merging, which was consistent with our planned workflow. The technology stack matched our plan: Python for the backend (Flask), React/TypeScript for the frontend, YOLO with TensorRT for computer vision, and an STM32-based custom PCB for gimbal control, all deployed on the Jetson Orin Nano.

6.2 What Went Well?

- **CI/CD pipeline:** Our GitHub Actions setup for automated linting and testing caught issues early and maintained code quality throughout the project. Every pull request was required to pass CI checks before merging.
- **Structured sprint process:** Two-week sprints with clear deliverables kept the team on track and made progress visible to all stakeholders.
- **PR-based code review:** Requiring at least one approving review before merging improved code quality and knowledge sharing across team members. Several bugs were caught during review that would have been difficult to debug later.
- **Issue-driven feedback tracking:** Using GitHub Issues to track all feedback items made it straightforward to ensure every piece of feedback was addressed and traceable.

6.3 What Went Wrong?

- **Camera sensor change:** The mid-project switch to a different camera sensor required significant rework of the CAD model, updated calibration parameters, and revised latency measurements. This consumed approximately two weeks of effort that had not been planned for.
- **Integration complexity underestimated:** Integrating the custom PCB with the Jetson, the CV pipeline with the livestream, and the frontend with the backend all proved more complex than anticipated. Serial communication timing issues between the STM32 and Jetson required substantial debugging effort.
- **Limited field testing:** Due to weather constraints and schedule pressure, we were unable to conduct as many outdoor field tests as we had planned. Most testing was done in controlled indoor environments.

6.4 What Would you Do Differently Next Time?

- **Start hardware integration earlier:** We would begin hardware-software integration testing in the first month rather than waiting until the individual components were more mature. Early integration would have surfaced the serial communication issues sooner.
- **Allocate more time for field testing:** We would schedule regular outdoor testing sessions starting from Rev 0, rather than treating field testing as a final validation step.
- **Component sourcing contingency:** We would identify backup components for critical hardware (especially sensors) at the outset of the project to reduce the impact of mid-project changes.
- **Documentation-as-you-go:** While our documentation was ultimately comprehensive, several documents required significant last-minute updates. Maintaining documents in parallel with development would distribute the workload more evenly.

7 Ethics and Equity (LO18)

Our team applied principles of equity and universal design throughout the project to ensure the system is accessible and fair to all stakeholders.

- **Bilingual user interface:** The operator interface supports both English and French, reflecting Canada’s official bilingualism and ensuring accessibility for team members and users who are more comfortable in either language.
- **Colour-blind safe indicators:** After peer feedback (Issue #346) identified that green “Pass” indicators were not distinguishable for users with red-green colour vision deficiency, we replaced all green status indicators with teal, achieving a contrast ratio of at least 4.5:1 (WCAG AA compliance). All status information is also conveyed through text labels, not colour alone.
- **Outdoor-readable single-page UI:** The interface was designed as a single-page application with large, high-contrast elements optimized for use on tablets in bright outdoor conditions. This design decision ensures usability for operators who may be wearing sunglasses or gloves.
- **Confirmation prompts for dangerous actions:** To prevent accidental harm—particularly relevant given the gimbal’s mechanical movement—all potentially dangerous operations (gimbal reset, firmware flash, emergency stop override) require explicit confirmation. This protects both the operator and bystanders.
- **Open-source licensing:** The project is released under the MIT license, ensuring equitable access to the technology regardless of institutional affiliation or budget. Any rocketry team, hobbyist, or researcher can use, modify, and redistribute the system at no cost.

- **Privacy considerations:** The system is designed for tracking rockets, not people. We intentionally trained the YOLO model exclusively on rocket imagery and did not include any person-detection capabilities. The system operates on a local network with no cloud data transmission, minimizing privacy risks for bystanders at launch events.

8 Reflection on Capstone

8.1 Which Courses Were Relevant

Several courses from our undergraduate curriculum provided foundational knowledge that was directly applicable to the RoCam project:

- **SFWRENG 3K04 – Software Design III:** This course provided the foundation for our modular software architecture, including the use of design patterns and the module decomposition reflected in our Module Guide.
- **SFWRENG 2C03 – Data Structures and Algorithms:** Knowledge of efficient data structures was essential for the real-time tracking pipeline, particularly for managing detection queues and frame buffers.
- **SFWRENG 3DX4 – Signals and Systems:** Understanding of signal processing concepts was directly applicable to the GStreamer video pipeline design and the PID control loop for gimbal stabilization.
- **CAS 741 – Development of Scientific Computing Software:** This graduate-level course informed our approach to rigorous documentation, including the SRS, MG, MIS, and VnV Plan/Report structure used throughout the project.

8.2 Knowledge/Skills Outside of Courses

The capstone project required us to learn several technologies and skills that were not covered in our coursework:

- **TensorRT optimization:** None of our courses covered NVIDIA’s TensorRT framework. We learned to convert YOLO models to TensorRT engines, perform INT8 calibration, and benchmark inference performance on the Jetson Orin Nano.
- **GStreamer pipelines:** Building low-latency video encoding and decoding pipelines using GStreamer with hardware-accelerated codecs required substantial self-directed learning, as no course covered multimedia framework programming.
- **PCB design:** Designing the custom STM32-based PCB required learning schematic capture, PCB layout, design rule checking, and manufacturing processes (Xiaotian). This included learning KiCad and understanding EMI/EMC considerations for motor control circuits.
- **Real-time systems programming:** While courses touched on concurrency, the practical challenges of building a real-time system with strict latency requirements (<120 ms end-to-end) required learning about process isolation, shared memory IPC, and Linux real-time scheduling.
- **React and TypeScript frontend development:** The frontend was built with React and TypeScript, neither of which was covered in our curriculum. We learned component-based UI architecture, state management, and responsive design for the operator interface.
- **Embedded firmware development:** Writing firmware for the STM32 microcontroller, including UART communication protocols and PID motor control, required learning the STM32 HAL library and embedded C programming practices.